



Cairo University
Faculty of Engineering

Department of Computer
Engineering



ELC 325B – Spring 2025

Digital Communications

Assignment #3

Submitted to

Dr. Mai Badaway

Dr. Hala Abdelkader

Eng. Mohammed Khalid

Submitted by

Name	Sec	BN
Ahmed Hamdy Mohammed	1	4
Abdelrahman Mostafa Gomaa	1	31

Contents

1	Gram-Schmidt Orthogonalization	3
2	Signal Space Representation	4
3	Effect of AWGN on signal space representation	5
3.1	Case (1): $\frac{E}{\sigma^2} = 5dB$	5
3.2	Case (2): $\frac{E}{\sigma^2} = 0dB$	5
3.3	Case (3): $\frac{E}{\sigma^2} = 10dB$	6
A	Appendix	7
1	Code	7

List of Figures

1	$\Phi_1(t)$ after using GM_Bases	3
2	$\Phi_2(t)$ after using GM_Bases	3
3	Signal Space of $s_1(t), s_2(t)$	4
4	Signal Space of $s_1(t), s_2(t)$ with $\frac{E}{\sigma^2} = -5dB$	5
5	Signal Space of $s_1(t), s_2(t)$ with $\frac{E}{\sigma^2} = 0dB$	5
6	Signal Space of $s_1(t), s_2(t)$ with $\frac{E}{\sigma^2} = 10dB$	6

1 Gram-Schmidt Orthogonalization

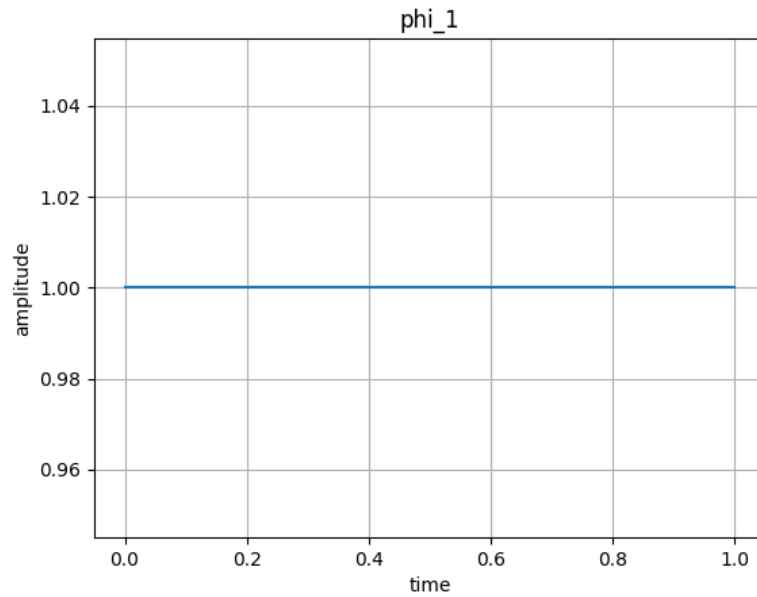


Figure 1: $\Phi_1(t)$ after using GM_Bases

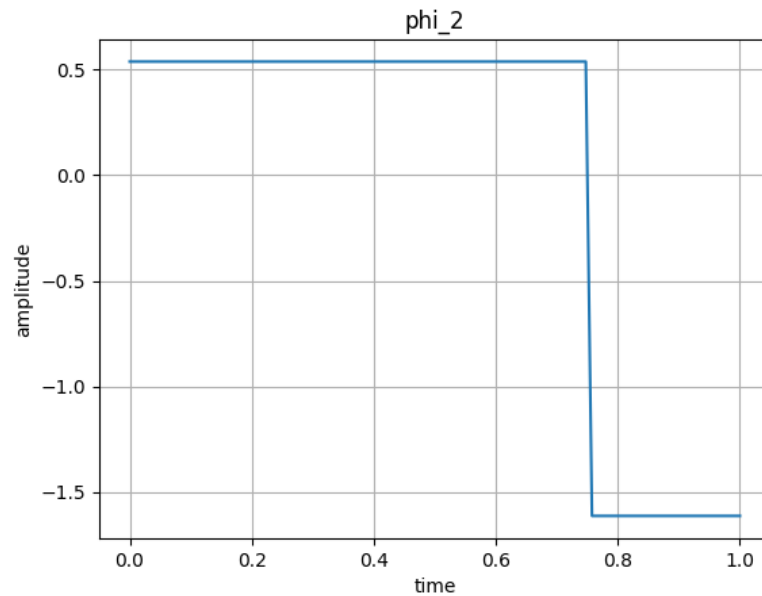


Figure 2: $\Phi_2(t)$ after using GM_Bases

Comment

The simulation results of Gram-Schmidt orthogonalization matches our expectation. $\Phi_1(t)$ is the normalization of $s_1(t)$ and $\Phi_2(t)$ is orthogonal. If we perform a dot product on them, the result will be zero.

2 Signal Space Representation

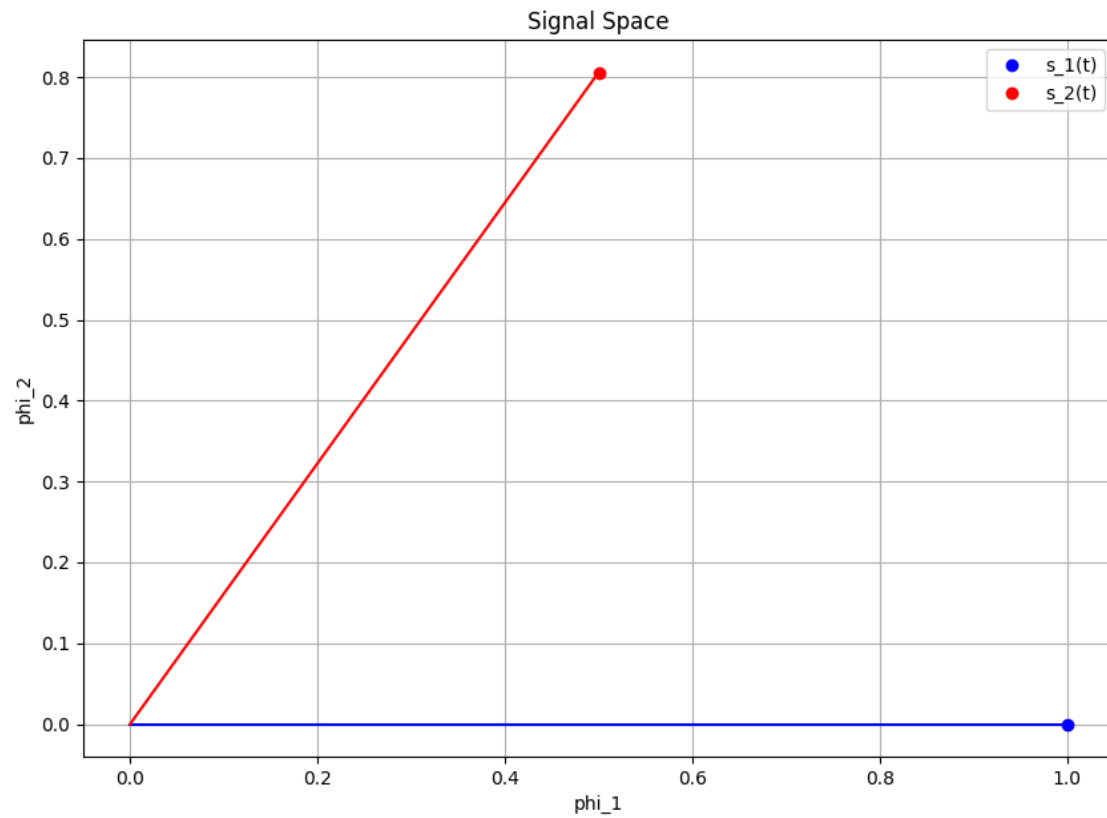


Figure 3: Signal Space of $s_1(t), s_2(t)$

Comment

As shown that $s_1(t)$ aligns fully with $\Phi_1(t)$, resulting in a single nonzero coefficient for that basis. Conversely, $s_2(t)$ projects onto both $\Phi_1(t)$ and $\Phi_2(t)$, yielding two nonzero components.

3 Effect of AWGN on signal space representation

3.1 Case (1): $\frac{E}{\sigma^2} = 5dB$

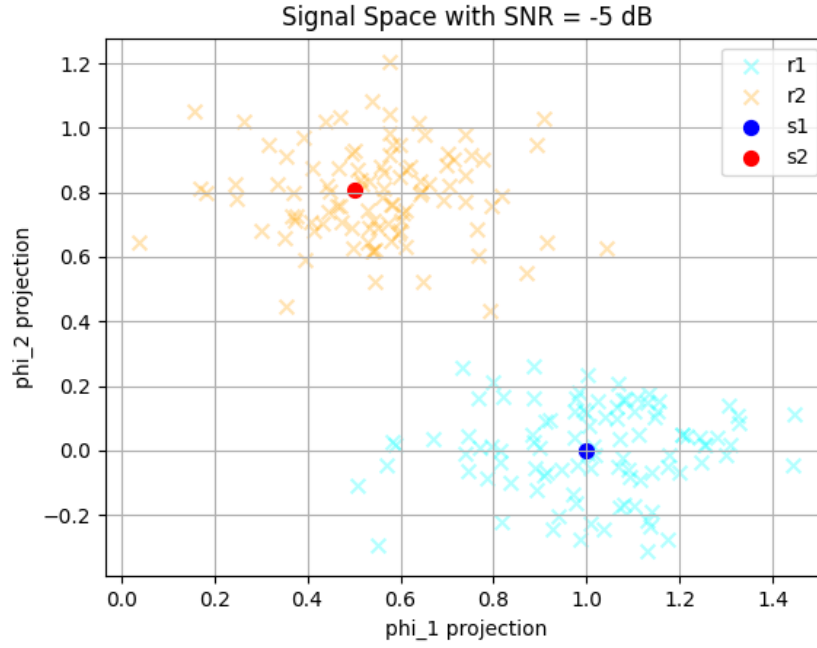


Figure 4: Signal Space of $s_1(t), s_2(t)$ with $\frac{E}{\sigma^2} = -5dB$

3.2 Case (2): $\frac{E}{\sigma^2} = 0dB$

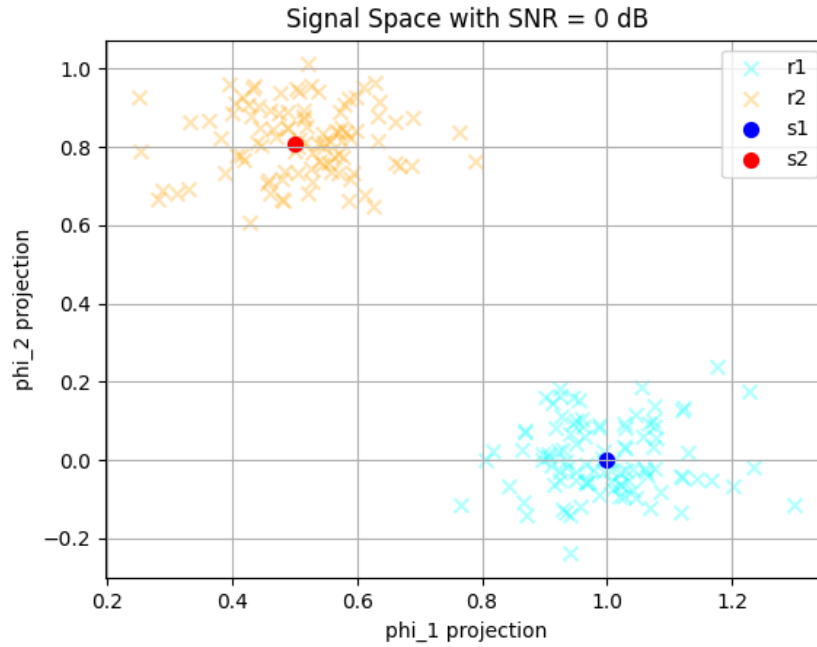


Figure 5: Signal Space of $s_1(t), s_2(t)$ with $\frac{E}{\sigma^2} = 0dB$

3.3 Case (3): $\frac{E}{\sigma^2} = 10dB$

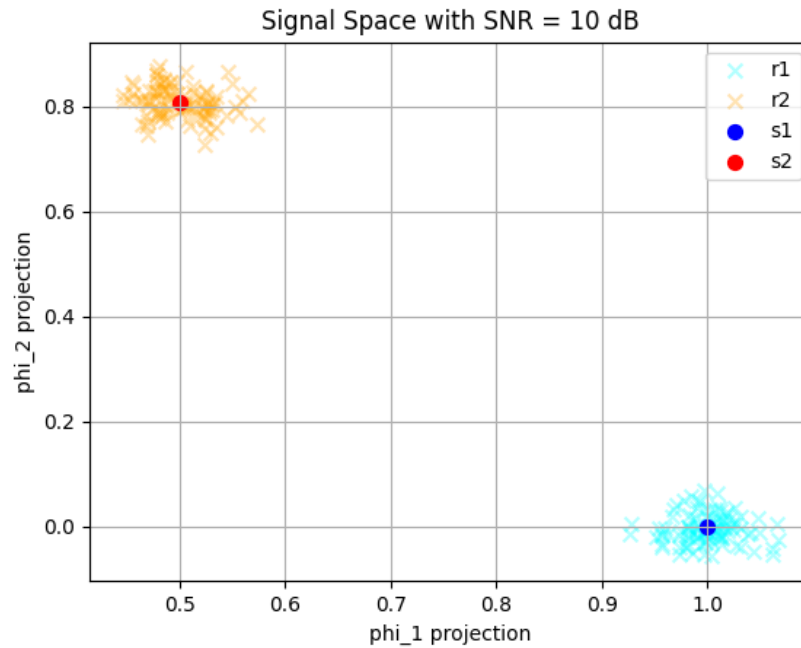


Figure 6: Signal Space of $s_1(t), s_2(t)$ with $\frac{E}{\sigma^2} = 10dB$

Comment

From the three cases, it is obvious that as $\frac{E}{\sigma^2}$ increases, the noisy points tend to concentrate more tightly around the ideal locations.

How does the noise affect the signal space? Does the noise effect increase or decrease with increasing σ^2 ?

Noise causes the signal points in signal space to scatter. When σ^2 increases, the effect of noise increases, and the points become more spread out. So, a smaller σ^2 improves the distinction between the two signals in signal space, and reduces the probability of detection error.

Appendix A

Appendix

1 Code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Signal Processing Functions
5 def magnitude(signal, samples_number):
6     return np.sqrt(np.dot(signal, signal)/samples_number)
7
8 def GM_Bases(s1, s2, samples_number):
9     # calculate phi_1 as normalized of first signal
10    phi_1 = s1 / np.sqrt(magnitude(s1, samples_number))
11
12    # calculate phi_2 as orthogonal to phi_1
13    phi_2 = s2 - phi_1 * np.dot(phi_1, s2) / samples_number
14    phi_2_norm = np.sqrt(magnitude(phi_2, samples_number))
15
16    # special case: s1 and s2 have one basis function
17    if phi_2_norm == 0:
18        phi_2 = np.zeros_like(phi_2)
19    else:
20        phi_2 /= phi_2_norm
21
22    return phi_1, phi_2
23
24 def signal_space(s, phi1, phi2, samples_number):
25     v1 = np.dot(s, phi1)/samples_number
26     v2 = np.dot(s, phi2)/samples_number
27     return {"v1": v1, "v2": v2}
28
29 def add_noise(s, E_No, samples_number):
30     E = magnitude(s, samples_number)
31     E_No = 10 ** (E_No / 10)
32     sigma_squared = E/E_No
33     noise = np.random.normal(0, np.sqrt(sigma_squared), samples_number)
34     return s + noise
35
36 # Plotting Functions
37 def plot_signal(signal, t, title):
38     plt.figure()
39     plt.plot(t, signal)
40     plt.title(title)
41     plt.xlabel('time')
42     plt.ylabel('amplitude')
43     plt.grid()
44     plt.show()
45
46 def plot_signal_space(signals, titles, colors):
47     plt.figure(figsize=(10,7))
48     for i in range(len(signals)):
49         plt.plot(signals[i]["v1"], signals[i]["v2"], f'{colors[i]}o', label=f'{titles[i]}")
```



```

50     plt.plot([0, signals[i]["v1"]], [0, signals[i]["v2"]], color=colors[i])
51     plt.title('Signal Space')
52     plt.xlabel("phi_1")
53     plt.ylabel("phi_2")
54     plt.legend()
55     plt.grid()
56     plt.show()
57
58 def plot_signal_space_result(r1_org, r2_org, noisy_r1_points, noisy_r2_points, SNR):
59     plt.figure()
60     plt.title(f'Signal Space with SNR = {SNR} dB')
61
62     r1_points = np.array([(p['v1'], p['v2']) for p in noisy_r1_points])
63     r2_points = np.array([(p['v1'], p['v2']) for p in noisy_r2_points])
64     r1_v1s, r1_v2s = r1_points.T
65     r2_v1s, r2_v2s = r2_points.T
66
67     plt.scatter(r1_v1s, r1_v2s, label="r1", color='cyan', marker='x', alpha=0.3, s=50)
68     plt.scatter(r2_v1s, r2_v2s, label="r2", color='orange', marker='x', alpha=0.3, s=50)
69     plt.scatter(r1_org["v1"], r1_org["v2"], label="s1", color='blue', s=50)
70     plt.scatter(r2_org["v1"], r2_org["v2"], label="s2", color='red', s=50)
71
72     plt.legend()
73     plt.grid(True)
74     plt.xlabel('phi_1 projection')
75     plt.ylabel('phi_2 projection')
76     plt.show()
77
78 # Experiments
79 samples_number = 100
80 t = np.linspace(0, 1, samples_number)
81
82 # Generate signals and plot them
83 s1 = np.ones(samples_number)
84 s2 = np.ones(samples_number)
85 s2[75:] = -1
86 plot_signal(s1, t, 's_1(t)')
87 plot_signal(s2, t, 's_2(t)')
88
89 # Calculate and plot basis functions
90 phi_1, phi_2 = GM_Bases(s1, s2, samples_number)
91 plot_signal(phi_1, t, 'phi_1')
92 plot_signal(phi_2, t, 'phi_2')
93
94 # Plot signal space representation
95 signal_1 = signal_space(s1, phi_1, phi_2, samples_number)
96 signal_2 = signal_space(s2, phi_1, phi_2, samples_number)
97 plot_signal_space([signal_1, signal_2], ["s_1(t)", "s_2(t)"], ['b', 'r'])
98
99 # Add noise and plot results for different SNR values
100 SNR_values = [-5, 0, 10]
101 for SNR in SNR_values:
102     r1 = signal_space(s1, phi_1, phi_2, samples_number)
103     r2 = signal_space(s2, phi_1, phi_2, samples_number)
104
105     noisy_r1_points = []
106     noisy_r2_points = []
107     for _ in range(samples_number):
108         r1_noisy = add_noise(s1, SNR, samples_number)
109         r2_noisy = add_noise(s2, SNR, samples_number)
110         r1__noisy_proj = signal_space(r1_noisy, phi_1, phi_2, samples_number)
111         r2__noisy_proj = signal_space(r2_noisy, phi_1, phi_2, samples_number)
112         noisy_r1_points.append(r1__noisy_proj)
113         noisy_r2_points.append(r2__noisy_proj)
114
115     plot_signal_space_result(r1, r2, noisy_r1_points, noisy_r2_points, SNR)

```

Listing A.1: project code